Supplementary Material 3: Interoperability workflow

Using *chromExtract()* to import external peak tables and run quality assessment

Philippine Louail Laurent Gatto Sebastian Gibb Johannes Rainer

2026-06-02

Table of contents

Supplementary Material for:

Chromatograms: A modular infrastructure for scalable chromatographic data handling in R

Philippine Louail[1,2,*], Laurent Gatto[3], Sebastian Gibb[4], Johannes Rainer[1]

[1] Institute for Biomedicine, Eurac Research, Bolzano, Italy [2] Chair for Bioinformatics, Friedrich-Schiller-University Jena, 07743 Jena, Germany [3] Computational Biology and Bioinformatics, de Duve Institute, UCLouvain, Brussels, Belgium [4] Department of Anaesthesiology and Intensive Care, University Medicine Greifswald, Greifswald, Germany

[*] Corresponding author: philippine.louail@outlook.com

Motivation

In practice, peak picking and downstream analysis are often performed by different tools. A peak list produced by *MZmine*, *MS-DIAL*, *xcms*, or a custom script cannot usually be reused by a different package without format conversion or wrapper code.

`Chromatograms` addresses this with *chromExtract()* - an R function that accepts a plain peak table (a `data.frame` containing m/z and retention-time boundaries together with feature identifiers) produced by *any* upstream peak-picking software, matches it against the raw mzML data, and returns the corresponding chromatograms as a `Chromatograms` object that is immediately usable in any R workflow. The only contract is the column structure of the input `data.frame`; the software that produced the peak list is irrelevant.

Two architectural choices make this possible. First, *chromExtract()* relies on the *ChromBackendSpectra* backend, which keeps a reference to the underlying `Spectra` object instead of summarising spectra into retention-time / intensity traces. The full per-spectrum m/z × intensity arrays therefore remain accessible, and *chromExtract()* can filter by m/z range when building the extracted-ion chromatograms - a step that would be impossible on a backend that had already collapsed the m/z dimension. Second, because *ChromBackendSpectra* delegates raw-data I/O to `Spectra`, *chromEx-*

tract() inherits support for every backend that *Spectra* supports (mzML via *MsBackendMzR*, in-memory, third-party backends, etc.).

This document walks through the full workflow:

1. Create a generic peak table CSV that mimics the output of an external peak-picking tool.
2. Import it into R and match it against raw mzML data.
3. Extract the corresponding chromatograms with *chromExtract()*.
4. Verify that the external metadata are preserved on the resulting `Chromatograms` object and pass it to *MsQuality::calculateMetrics()* for automated quality assessment.

```
library(Chromatograms)
library(Spectra)
library(MsBackendMetaboLights)
library(dplyr)
library(ggplot2)
library(MsQuality)
```

System Information

```
knitr::kable(
  data.frame(
    Item  = c("CPU Cores", "R Version", "Platform", "Chromatograms Version"),
    Value = c(
      parallel::detectCores(),
      R.version$version.string,
      R.version$platform,
      as.character(packageVersion("Chromatograms"))
    )
  ),
  caption = "System information"
)
```

Table 1: System information

Item	Value
CPU Cores	20
R Version	R version 4.6.0 (2026-04-24 ucrt)
Platform	x86_64-w64-mingw32
Chromatograms Version	1.3.0

1. Create a generic peak table

In a real workflow this CSV would come from any peak-picking software - *MZmine*, *MS-DIAL*, *xcms*, a custom Python script, or a commercial vendor export. The only requirement is that the file carries columns defining the m/z and RT boundaries of each feature.

Here we **derive realistic targets from the raw data** by scanning a few representative spectra for the most intense m/z bins and defining narrow EIC windows around them. This ensures the

extracted chromatograms contain actual signal.

Table 2: Loaded data summary

Item	Value
Files	10
Total spectra	17210
RT range (s)	0.3 - 480.2

Table 3: Simulated external peak boundary export

feature_id	compound_name	adduct	mz_min	mz_max	rt_min_s	rt_max_s
FT_0001	Compound_A	[M-H]-	132.052	132.152	120.247	360.195
FT_0002	Compound_B	[M+FA-H]-	112.846	112.946	120.247	360.195
FT_0003	Compound_C	[M-H]-	139.069	139.169	120.247	360.195
FT_0004	Compound_D	[M+FA-H]-	290.797	290.897	120.247	360.195
FT_0005	Compound_E	[M-H]-	280.769	280.869	120.247	360.195
FT_0006	Compound_F	[M+FA-H]-	336.641	336.741	120.247	360.195
FT_0007	Compound_G	[M-H]-	114.843	114.943	120.247	360.195
FT_0008	Compound_H	[M+FA-H]-	155.992	156.092	120.247	360.195
FT_0009	Compound_I	[M-H]-	270.740	270.840	120.247	360.195
FT_0010	Compound_J	[M+FA-H]-	346.670	346.770	120.247	360.195
FT_0011	Compound_K	[M-H]-	272.737	272.837	120.247	360.195
FT_0012	Compound_L	[M+FA-H]-	282.767	282.867	120.247	360.195
FT_0013	Compound_M	[M-H]-	338.639	338.739	120.247	360.195
FT_0014	Compound_N	[M+FA-H]-	504.564	504.664	120.247	360.195
FT_0015	Compound_O	[M-H]-	334.644	334.744	120.247	360.195
FT_0016	Compound_P	[M+FA-H]-	344.673	344.773	120.247	360.195
FT_0017	Compound_Q	[M-H]-	514.593	514.693	120.247	360.195
FT_0018	Compound_R	[M+FA-H]-	494.535	494.635	120.247	360.195
FT_0019	Compound_S	[M-H]-	943.585	943.685	120.247	360.195
FT_0020	Compound_T	[M+FA-H]-	262.708	262.808	120.247	360.195

2. Import and Prepare for *chromExtract()*

Read the CSV and reshape it into the format expected by *chromExtract()*. The function requires `mzMin`, `mzMax`, `rtMin`, `rtMax`, and a `by` column to match against the `Chromatograms` object - every other column is preserved as metadata on the output.

```
# Read the external peak table
imported <- read.csv(csv_path)

# Ensure sample_origins is defined (in case it wasn't captured from upstream)
if (!exists("sample_origins") || length(sample_origins) == 0) {
  sample_origins <- unique(dataOrigin(sps))
}
```

```r
# Reshape to Chromatograms convention
# We expand one row per sample file (cross-join features × files)
peak_input <- expand.grid(
  feature_idx = seq_len(nrow(imported)),
  dataOrigin  = sample_origins,
  stringsAsFactors = FALSE
) |>
  mutate(
    mzMin         = imported$mz_min[feature_idx],
    mzMax         = imported$mz_max[feature_idx],
    rtMin         = imported$rt_min_s[feature_idx],
    rtMax         = imported$rt_max_s[feature_idx],
    msLevel       = 1L,
    feature_id    = imported$feature_id[feature_idx],
    compound_name = imported$compound_name[feature_idx],
    adduct        = imported$adduct[feature_idx],
    source_software = imported$source_software[feature_idx]
  ) |>
  select(-feature_idx)

knitr::kable(
  data.frame(
    Item  = c("Total rows", "Features", "Files"),
    Value = c(nrow(peak_input), nrow(imported), length(sample_origins))
  ),
  caption = "Expanded peak input (features x files)"
)
```

Table 4: Expanded peak input (features x files)

Item	Value
Total rows	200
Features	20
Files	10

3. Extract Chromatograms

3.1 Create a base Chromatograms object from raw data

Passing a `Spectra` object to the `Chromatograms()` constructor produces an object backed by *ChromBackendSpectra*. This backend keeps a reference to the original `Spectra` rather than summarising the spectra into retention-time / intensity traces, so the full m/z × intensity arrays remain accessible per spectrum. Data are read lazily - nothing is loaded into memory until explicitly requested. This is what enables *chromExtract()*: with the m/z dimension preserved, the function can filter on m/z range and construct extracted-ion chromatograms (EICs) on the fly. A backend that had pre-summarised the data into TIC- or BPC-like traces would no longer carry enough information for m/z-based extraction.

```
# Create a Chromatograms object backed by ChromBackendSpectra.
# The Spectra object is kept by reference - the m/z dimension is preserved.
chr_base <- Chromatograms(sps, chromData = peak_input)

chr_base
```

```
Chromatographic data (Chromatograms) with 200 chromatograms in a ChromBackendSpectra backend:
  chromIndex msLevel mz
1         NA       1 NA
2         NA       1 NA
3         NA       1 NA
4         NA       1 NA
5         NA       1 NA
6         NA       1 NA
... 10 more  chromatogram variables/columns
... 2 peaksData variables

The Spectra object contains 17210 spectra
```

3.2 Alternative: using *chromExtract()* on an existing object

If a broad `Chromatograms` object covering the full RT/m/z range already exists, *chromExtract()* can be used to narrow it. This only works when the object is backed by *ChromBackendSpectra* (i.e. created from a `Spectra` object), because *chromExtract()* needs the per-spectrum m/z × intensity arrays to filter on m/z range. A `Chromatograms` object that has already been summarised into RT vs. intensity traces no longer carries the m/z dimension and cannot be used with *chromExtract()*.

```
# Create a broad Chromatograms (full RT range, per-file, per-msLevel).
# The backend is ChromBackendSpectra - the m/z dimension is preserved.
chr_broad <- Chromatograms(sps)

chr_broad
```

```
Chromatographic data (Chromatograms) with 10 chromatograms in a ChromBackendSpectra backend:
  chromIndex msLevel  mz
1         NA       1 Inf
2         NA       1 Inf
3         NA       1 Inf
4         NA       1 Inf
5         NA       1 Inf
6         NA       1 Inf
... 7 more  chromatogram variables/columns
... 2 peaksData variables

The Spectra object contains 17210 spectra
```

```
# Now extract specific regions using our peak table.
# chromExtract() matches by the columns specified in `by`.
# peak.table must have: rtMin, rtMax, mzMin, mzMax, and the `by` columns.
```